

Intelligenza Artificiale

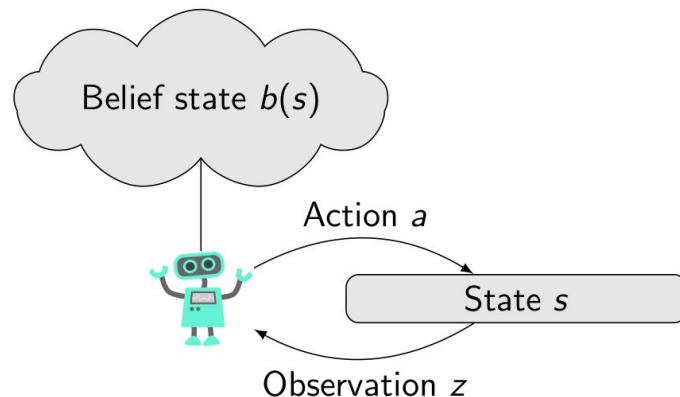
“Markov Decision Processes (part 2)”

Nicola Bellotto

A.A. 2025/2026

Lecture outline

- Policy iteration algorithm
- Partially observable MDPs
- Belief state
- POMDP decision cycle
- Optimal policy
- Application example



Slides partly based on Russell & Norvig instructors material <http://aima.cs.berkeley.edu/instructors.html>

Policy iteration algorithm

- Alternates the following two steps beginning from some initial policy π_0 :
 1. **Policy evaluation:** given a fixed policy π_i , calculate $U_i = U^{\pi_i}$, i.e. utility of each state if π_i were used
 2. **Policy improvement:** using U_i , calculate a new policy π_{i+1} that maximises the expected utility
- The algorithm terminates when the policy improvement step yields no change in the utilities

```
function POLICY-ITERATION(mdp) returns a policy
  inputs: mdp, an MDP with states  $S$ , actions  $A(s)$ , transition model  $P(s' | s, a)$ 
  local variables:  $U$ , a vector of utilities for states in  $S$ , initially zero
                   $\pi$ , a policy vector indexed by state, initially random

  repeat
     $U \leftarrow \text{POLICY-EVALUATION}(\pi, U, \textit{mdp})$ 
     $\textit{unchanged?} \leftarrow \text{true}$ 
    for each state  $s$  in  $S$  do
       $a^* \leftarrow \underset{a \in A(s)}{\text{argmax}} \text{ Q-VALUE}(\textit{mdp}, s, a, U)$ 
      if  $\text{Q-VALUE}(\textit{mdp}, s, a^*, U) > \text{Q-VALUE}(\textit{mdp}, s, \pi[s], U)$  then
         $\pi[s] \leftarrow a^*$ ;  $\textit{unchanged?} \leftarrow \text{false}$ 
  until  $\textit{unchanged?}$ 
  return  $\pi$ 
```

Policy iteration algorithm

- How to do **POLICY-EVALUATION**?

- At the i^{th} iteration, the policy π_i specifies the action $\pi_i(s)$ in state s

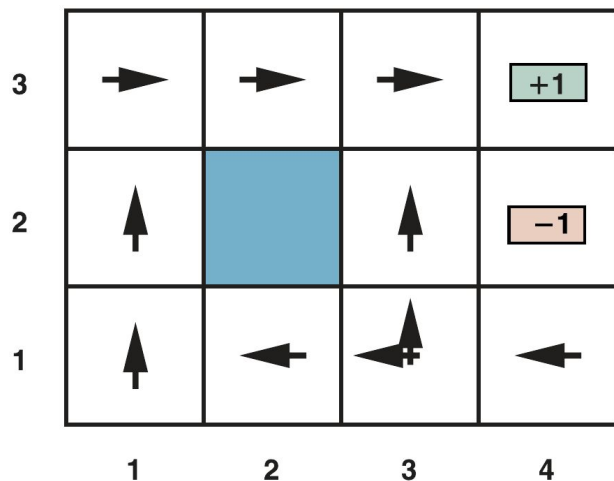
$$U_i(s) = \sum_{s'} P(s' | s, \pi_i(s)) [R(s, \pi_i(s), s') + \gamma U_i(s')]$$

- Basically a simplified version of Bellman equation relating the utility of s (with π_i) to those of its neighbors
 - no “**max**” operator $\Rightarrow$ linear equations $\Rightarrow$ can be solved by fast linear algebra!

```
function POLICY-ITERATION(mdp) returns a policy
  inputs: mdp, an MDP with states  $S$ , actions  $A(s)$ , transition model  $P(s' | s, a)$ 
  local variables:  $U$ , a vector of utilities for states in  $S$ , initially zero
                    $\pi$ , a policy vector indexed by state, initially random

  repeat
     $U \leftarrow \text{POLICY-EVALUATION}(\pi, U, \textit{mdp})$ 
    unchanged?  $\leftarrow$  true
    for each state  $s$  in  $S$  do
       $a^* \leftarrow \underset{a \in A(s)}{\text{argmax}} \text{ Q-VALUE}(\textit{mdp}, s, a, U)$ 
      if  $\text{Q-VALUE}(\textit{mdp}, s, a^*, U) > \text{Q-VALUE}(\textit{mdp}, s, \pi[s], U)$  then
         $\pi[s] \leftarrow a^*$ ; unchanged?  $\leftarrow$  false
  until unchanged?
  return  $\pi$ 
```

Policy iteration – example



- Let's assume the policy π_i in the figure

i.e. $\pi_i(1, 1) = Up$, $\pi_i(1, 2) = Up$, $\pi_i(1, 3) = Right$, ...

- The utilities of POLICY-EVALUATION are

$$U_i(1, 1) = 0.8[-0.04 + U_i(1, 2)] + 0.1[-0.04 + U_i(2, 1)] + 0.1[-0.04 + U_i(1, 1)]$$

$$U_i(1, 2) = 0.8[-0.04 + U_i(1, 3)] + 0.2[-0.04 + U_i(1, 2)]$$

$$U_i(1, 3) = 0.8[-0.04 + U_i(2, 3)] + 0.1[-0.04 + U_i(1, 2)] + 0.1[-0.04 + U_i(1, 3)]$$

...

Value iteration vs Policy iteration



- Value iteration

- simpler and faster iterations, but more are needed to converge to final utilities U^*
 - also, an additional step is required to extract best actions from U^*
- intermediate results are not useful
- good for small state spaces

- Policy iteration

- more complex iterations, but less are needed to converge to final policy π^*
- intermediate results are usable policies, albeit non-optimal
- good for large state spaces



Class of Markov models

Markov models	Passive (observations only)	Active (actions possible)
State fully observable	Markov chain	MDPs Markov Decision process
State partially observable	HMM Hidden Markov Model	POMDP Partially Observable MDP

Partially Observable MDPs

- **Partially observable MDPs (POMDPs)**
- MDP = [transition model $P(s' | s, a)$; actions $A(s)$; reward function $R(s, a, s')$]
- ➔ POMDPs are essentially MDPs with **sensor model** $P(e | s)$
- How to solve them?
 - 💡 Instead of considering real (hidden) states s_i , define a **belief state** b that includes all the probabilities of being in s_i – i.e. a **distribution** of probabilities $P(s_i) \equiv b(s_i)$
 - Then solve as normal MDP with observable **belief states**

Belief state – example

1/9	1/9	1/9	0
1/9		1/9	0
1/9	1/9	1/9	1/9

Belief state at time $t = 1$

$\langle 1/9, 1/9, 1/9, 1/9, 1/9, 1/9, 0, 1/9, 1/9, 1/9, 0 \rangle$

$$b_1([1,1]) = 1/9 \quad b_1([2,1]) = 1/9 \quad b_1([1,2]) = 1/9$$

1/9	1/9	1/9	0
1/18		1/9	0
2/9	1/18	1/9	1/9

Belief state at time $t = 2$

$\langle 2/9, 1/18, 1/9, 1/9, 1/18, 1/9, 0, 1/9, 1/9, 1/9, 0 \rangle$

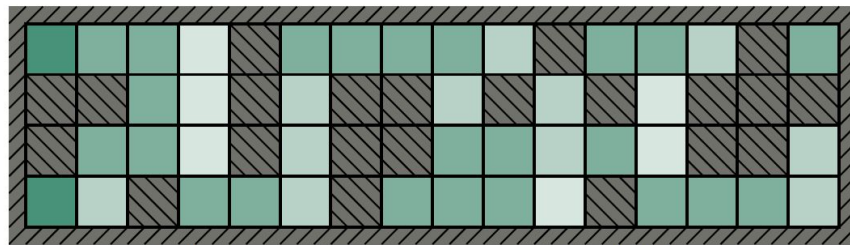
$$b_2([1,1]) = 2/9 \quad b_2([2,1]) = 1/18 \quad b_2([1,2]) = 1/18$$

Example: robot navigation

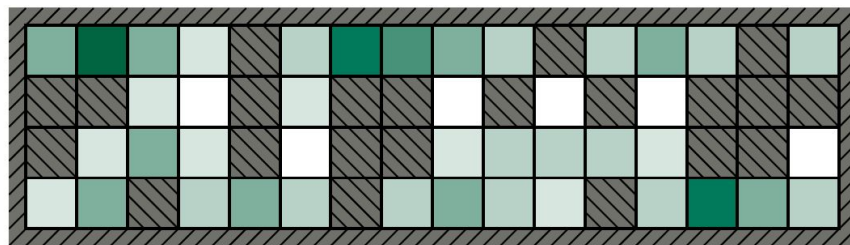
- Each probability distribution over all possible cells (actual states) is a particular belief state
- After every robot movement (action) new observations are used to update the belief



$t = 1$



$t = 2$

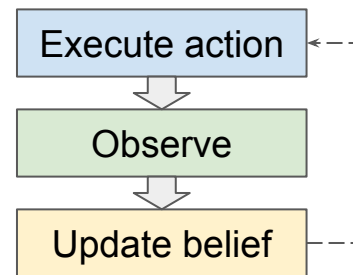


POMDP decision cycle

- The optimal action depends only on the agent's current belief state

- Steps of decision cycle of a POMDP agent

1. Given the current belief state b , execute the action $a = \pi^*(b)$
2. Observe the percept e
3. Update the current belief state b to the next b' , then repeat



- Can we formulate this as an MDP problem?

- We need belief estimation (for above step 3) + transition model and reward for the MDP

Belief estimation

- Given the usual Markov assumptions, the current belief can be estimated from the sequence of observations and actions executed so far
- Similar to **filtering**, except that the transition model includes also an action

$$b'(s') = \alpha P(e | s') \sum_s P(s' | s, a) b(s)$$

where s is the previous state, a is the action, s' is the current state, e is the current observation, and α is a normalization factor

- As in filtering, we can write this as a recursive operation for **belief estimation**

$$b' = \text{FORWARD}(b, a, e)$$

- This is a deterministic update because all the quantities, b , a , and e , are given inputs

Belief transition model



The **belief transition model** can be computed by summing over all the possible observations e

$$\begin{aligned} P(b'|b, a) &= \sum_e P(b'|e, a, b) \underline{P(e|a, b)} \\ &= \sum_e \underline{P(b'|e, a, b)} \sum_{s'} P(e|s') \sum_s P(s'|s, a) b(s) \end{aligned}$$

where $P(b'|e, a, b) = 1$ if $b' = \text{FORWARD}(b, a, e)$, and 0 otherwise (because it's deterministic)

The probability of an observation e after executing a from belief state b is given by summing over all the possible s' that the agent might actually reach

$$\begin{aligned} \underline{P(e|a, b)} &= \sum_{s'} P(e|a, s', b) P(s'|a, b) && \text{sensor Markov assumption} \\ &= \sum_{s'} P(e|s') P(s'|a, b) && = P(s|a, b) \\ &= \sum_{s'} P(e|s') \sum_s P(s'|s, a) b(s) && \text{1st order Markov assumption (+ action)} \end{aligned}$$

Reward function



- The **reward function** for belief-state transitions can be derived from the expected reward of the *real* state transition
 - i.e. the expected reward if the agent does a in belief state b is

$$\rho(b, a) = \sum_s b(s) \sum_{s'} P(s'|s, a) R(s, a, s')$$

- Together, $P(b' | b, a)$ and $\rho(b, a)$ define an observable MDP on the space of belief states

Optimal policy

- It can be shown that an optimal policy for this MDP is also optimal for the original POMDP
⇒ solving a POMDP on a (physical) state space can be reduced to solving an MDP on the corresponding belief-state space
- The main difference is that, even in case of finite state space, there are infinitely many belief state! (because each belief is a probability)
- One possibility is to discretize the belief state space, but this is not always possible...
- POMDPs can be computationally intractable to solve exactly, so **approximate solutions** must be used

Resources: <http://pomdp.org/> <https://github.com/Svalorzen/AI-Toolbox>

Application example

ActiVis: Active Vision with Human in the Loop for People with Vision Impairment

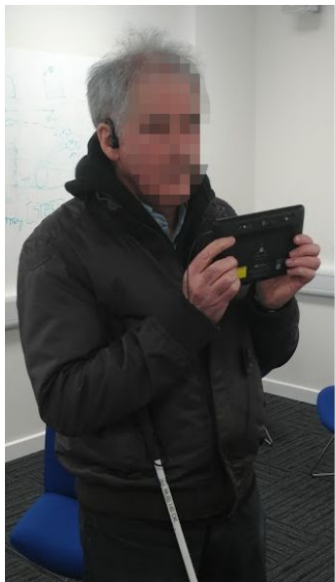


Google Faculty Research Award, 2015

<https://lcas.github.io/ActiVis/>

Object search for the visually impaired

- Assistive device for the VI to find objects in an unknown environment



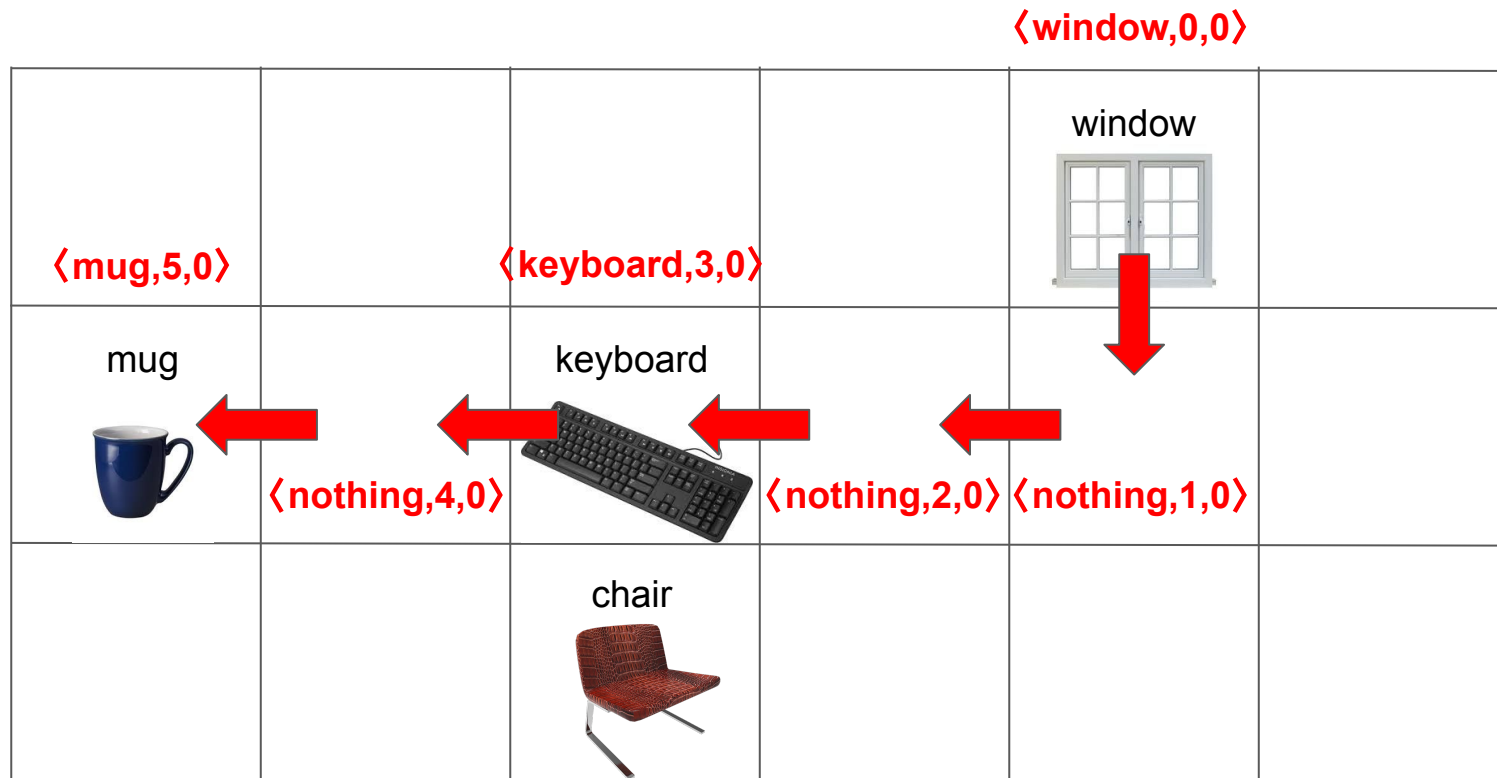
Camera-based object search

				window 	
mug 		keyboard 			
		chair 			

Camera-based object search

				 window	
 mug		 keyboard			
		 chair			

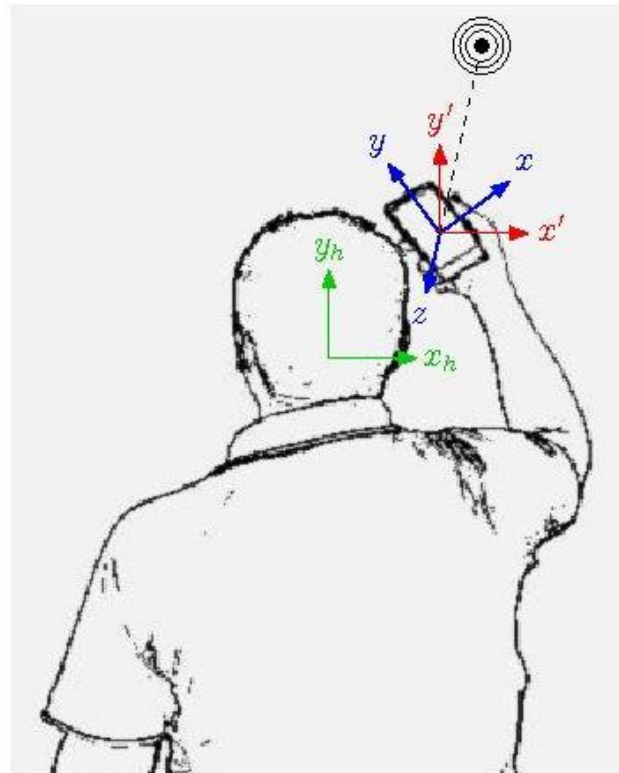
Active object search



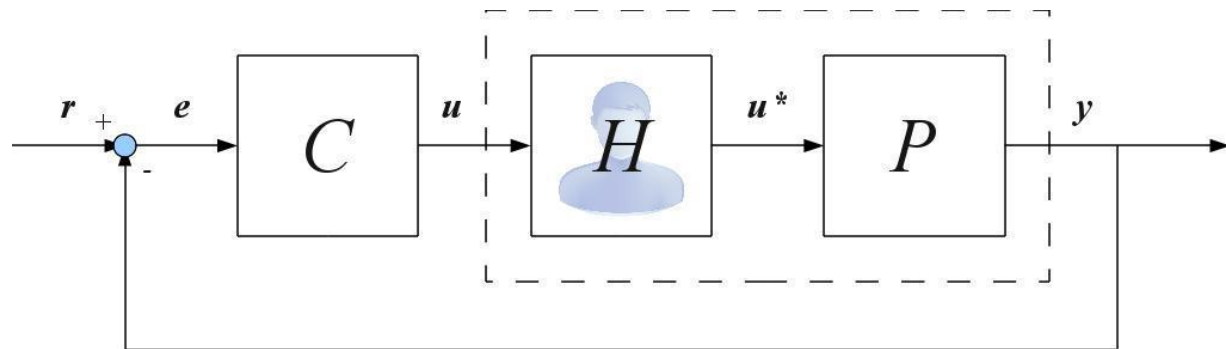
Active vision with human-in-the-loop



- Instead of electro-mechanical actuators, what if the camera is moved by a human?
- How do we “control” the hand, arm, body of the human to obtain the next best view?



Active vision with human-in-the-loop



r : reference (task-specific)

H : human

P : perception system (camera)

C : controller (POMDP)

u : control signal (user interface)

y : output (camera view/position/orientation)

User interface and camera perception

Audio-based instructions and sounds

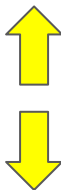
- Bone-conducting headset



- Spatialised stereo tone for pan instructions

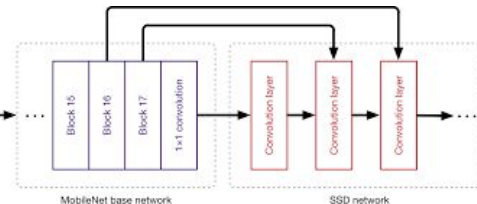


- Varying tone pitch for tilt instructions

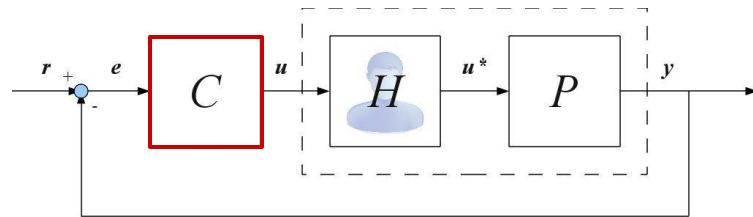


NN-based visual object detector

- MobileNetV2 + SSD-Lite
- Lightweight, suitable for mobile devices
- Trained with ~10k object samples (per class) on the Open Images dataset
- Produced a TensorFlow Lite model that is Android-compatible



POMDP controller



- POMDP model for the controller
 - states **S**: ⟨**o**bject in camera view, **n**umber of steps, **v**isited waypoint⟩
 - actions **A**: {**L**EFT, **R**IGHT, **U**P, **D**OWN}
 - transition **T**: probabilities $P(\mathbf{s}' \mid \mathbf{s}, \mathbf{a})$ of going to state $\mathbf{s}'$ starting from $\mathbf{s}$ and executing $\mathbf{a}$
 - sensor model **O**: observation probabilities from the confusion matrix of the object detector
 - reward **R**

Object classes

- Monitor
- Keyboard
- Mouse
- Desk
- Laptop
- Mug
- Window
- Lamp



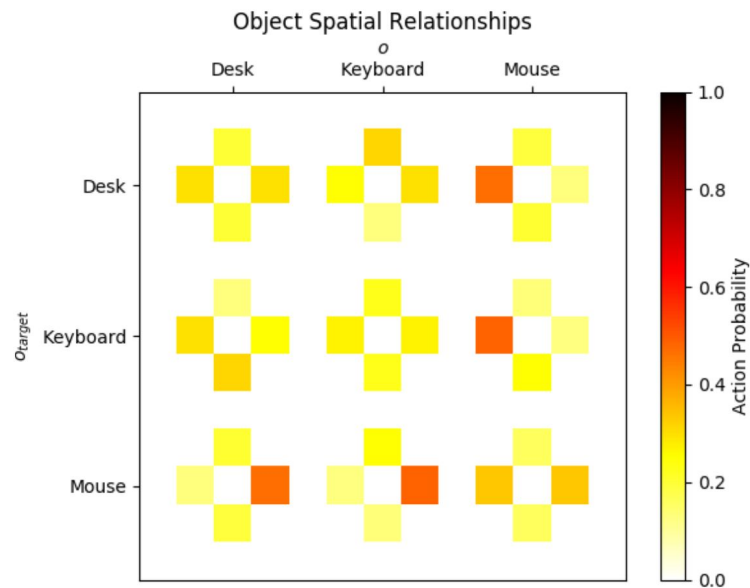
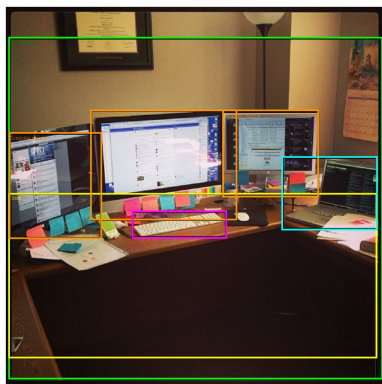
- Backpack
- Chair
- Couch
- Plant
- Telephone
- Whiteboard
- Door
- Nothing



Transition model

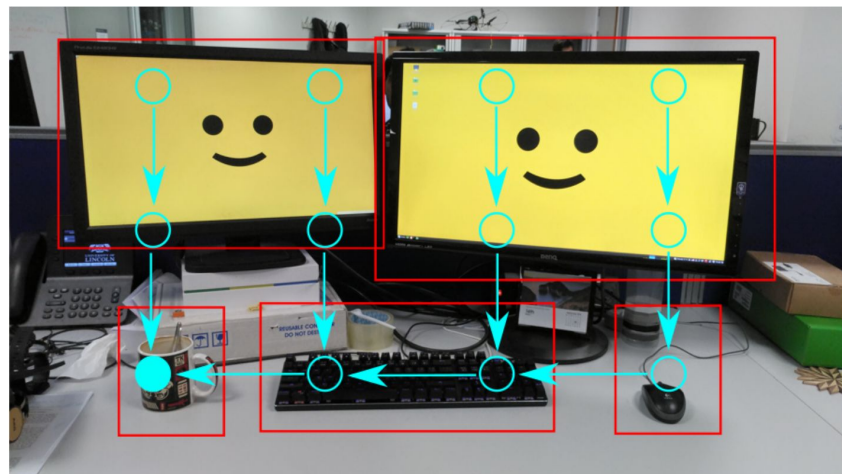
- Transition $\mathbf{T}$ learned from Open Images dataset*

* <https://storage.googleapis.com/openimages/web/index.html>



Policies

- Trained by a **Point-Based Value Iteration (PBVI)**
- 15 policies are generated, one for each target object (*terminal state*)



e.g. target = 'mug'

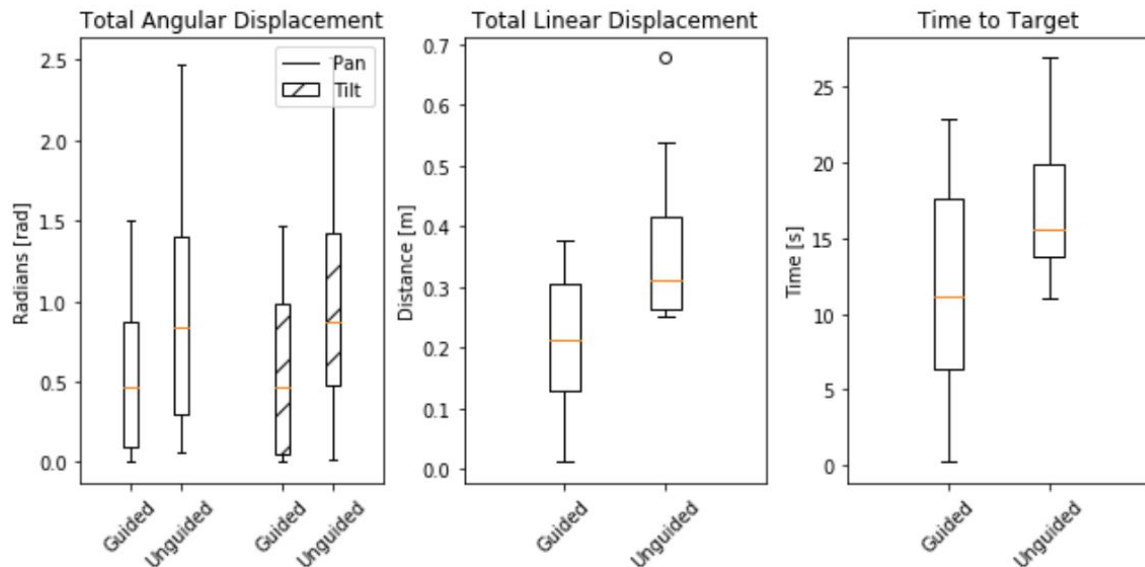
Experiments

10 participants

- 8 blindfolded
- 2 visually impaired



	Guided	Unguided	KW Statistic
TAR [%]	46 ± 13	54 ± 15	0.16
Time to target [s]	12.5 ± 11.9	17.2 ± 11.5	0.045
Pan Angle Displacement [rad]	0.68 ± 1.1	0.99 ± 1.2	0.029
Tilt Angle Displacement [rad]	0.68 ± 1.1	1.04 ± 1.2	0.011
Linear Displacement [rad]	0.23 ± 0.22	0.36 ± 0.22	0.012



Conclusions

- Suggested reading
 - Chapter 16, sec. 16.4 of Russell & Norvig
 - J. C. Lock et al. (2019) “ActiVis: Mobile Object Detection and Active Guidance for People with Visual Impairments“, International Conf. on Image Analysis and Processing (ICIAP).
https://link.springer.com/chapter/10.1007/978-3-030-30645-8_59
- Next lecture
Machine learning
- Questions?

